# Rockchip Developer Guide PCIE EP Standard Card APIs

ID: RK-KF-YF-C52

Release Version: V3.4.0

Release Date: 2026-03-20

Security Level: □Top-Secret □Secret □Internal ■Public

**DISCLAIMER**

THIS DOCUMENT IS PROVIDED “AS IS”. ROCKCHIP ELECTRONICS CO., LTD.(“ROCKCHIP”)DOES NOT PROVIDE ANY WARRANTY OF ANY KIND, EXPRESSED, IMPLIED OR OTHERWISE, WITH RESPECT TO THE ACCURACY, RELIABILITY, COMPLETENESS,MERCHANTABILITY, FITNESS FOR ANY PARTICULAR PURPOSE OR NON-INFRINGEMENT OF ANY REPRESENTATION, INFORMATION AND CONTENT IN THIS DOCUMENT. THIS DOCUMENT IS FOR REFERENCE ONLY. THIS DOCUMENT MAY BE UPDATED OR CHANGED WITHOUT ANY NOTICE AT ANY TIME DUE TO THE UPGRADES OF THE PRODUCT OR ANY OTHER REASONS.

**Trademark Statement**

"Rockchip", "瑞芯微", "瑞芯" shall be Rockchip’s registered trademarks and owned by Rockchip. All the other trademarks or registered trademarks mentioned in this document shall be owned by their respective owners.

**All rights reserved. ©2026. Rockchip Electronics Co., Ltd.**

Beyond the scope of fair use, neither any entity nor individual shall extract, copy, or distribute this document in any form in whole or in part without the written approval of Rockchip.

Rockchip Electronics Co., Ltd.

No.18 Building, A District, No.89, software Boulevard Fuzhou, Fujian,PRC

Website: www.rock-chips.com

Customer service Tel: +86-4007-700-590

Customer service Fax: +86-591-83951833

Customer service e-Mail: fae@rock-chips.com

## Preface

### Overview

This document explains the use of the RK PCIE standard EP development package and introduces the relevant APIs.

### Product Version

| Chipset | Kernel Version |
| --- | --- |
| All main chips supporting RK PCIe EP | Linux 6.1, 5.10, 4.19, RT-Thread v5.1.x, v4.1.x |

Note:

- Only RT-Thread support is implemented on the RK PCIe EP side

### Intended Audience

This document (this guide) is mainly intended for:

Technical support engineers

Software development engineers

### Revision History

| Version | Author | Date | Change Description |
| --- | --- | --- | --- |
| V1.0.0 | yp.xiao, Kever Yang, Jon Lin | 2023-03-07 | Initial version |
| V1.1.0 | yp.xiao | 2023-05-22 | Updated SDK API and some configuration instructions |
| V1.2.0 | yp.xiao | 2023-06-15 | Added EP card firmware compilation, burning, and OTA upgrade instructions<br>Added application exception instructions<br>Updated `struct pcie_dev_attr_st` structure to support timeout configuration, PCIE speed acquisition, RC DMA configuration. |
| v1.3.0 | Jon Lin | 2023-10-07 | Added pcie.bin instructions |
| v1.4.0 | yp.xiao, Jon Lin | 2023-10-23 | Updated SDK API<br>Added EP acceptance criteria section<br>Updated dual storage instructions<br>Updated PCIE BIN instructions |
| v1.5.0 | Jon Lin | 2024-05-23 | Updated PHY Mode settings |
| v1.6.0 | Jon Lin | 2024-09-18 | Added user-mode test demo - pcie_speed_test performance reference, added BAR1/5 support |
| v1.7.0 | yp.xiao, Jon Lin | 2025-01-24 | chips_connect development package added OSAL, compatible with Windows version |
| v1.8.0 | Jon Lin | 2025-02-12 | Optimized EP startup solution instructions, modified layout issues |
| v2.0.0 | Jon Lin | 2025-03-09 | Added RK chip interconnection independent power supply solution instructions |
| v3.0.0 | Jon Lin | 2025-05-14 | Adjuster note to EP card specifications, optimized EP Boot instructions, added data transfer principle instructions, added user layer multi-application instructions |
| v3.1.0 | Jon Lin | 2025-07-12 | Added RK1820 |
| v3.2.0 | Jon Lin | 2025-08-11 | Independently output Chips Connect APIs documentation |
| v3.3.0 | Jon Lin | 2025-08-15 | Add err_event, pcie_is_alive and hardware_reset apis |

| Version | Author | Date | Change Description |
| --- | --- | --- | --- |
| v3.4.0 | yp.xiao | 2026-03-20 | Add log setting, buff replacement, firmware upload and other new APIs; Update interface parameter types |

## Contents


Rockchip Developer Guide PCIE EP Standard Card APIs
1. Chips Connect API Reference
1.1 PCIe Linux Root Complex
1.1.1 rk_pcie_get_pci_bus_addresses
1.1.2 rk_pcie_device_init
1.1.3 rk_pcie_device_deinit
1.1.4 rk_pcie_task_create
1.1.5 rk_pcie_task_destroy
1.1.6 rk_pcie_boot_init
1.1.7 rk_pcie_boot_deinit
1.1.8 rk_pcie_boot_download_firmware
1.1.9 rk_pcie_boot_run
1.1.10 rk_pcie_get_ep_info
1.1.11 rk_pcie_get_user_cmd
1.1.12 rk_pcie_get_buff
1.1.13 rk_pcie_send_buff
1.1.14 rk_pcie_get_bar1_user_buffer
1.1.15 rk_pcie_get_bar5_user_buffer
1.1.16 rk_pcie_set_bar_inbound_cpu_addr
1.1.17 rk_pcie_release_buff
1.1.18 rk_pcie_get_buff_max_size
1.1.19 rk_pcie_get_ep_mode
1.1.20 Rescanning PCI Devices
1.1.21 rk_pcie_send_msg_to_ep_bus
1.1.22 rk_pcie_get_msg_for_task_id
1.1.23 rk_pcie_get_ep_obj_info
1.1.24 rk_pcie_get_ep_err_event_count
1.1.25 rk_pcie_is_alive
1.1.26 rk_pcie_device_hardware_reset
1.1.27 rk_pcie_set_log_level
1.1.28 rk_pcie_set_log_cb
1.1.29 rk_pcie_device_hardware_reset_sync
1.1.30 rk_pcie_is_device_hardware_reset
1.1.31 rk_pcie_boot_upload_firmware
1.1.32 rk_pcie_get_boot_state
1.1.33 rk_pcie_set_fw_info
1.1.34 rk_pcie_send_merge_msg_to_ep_bus
1.1.35 rk_pcie_replace_buff_all
1.1.36 rk_pcie_replace_buff
1.1.37 Data Types
1.1.37.1 RK_PCIE_HANDLES
1.1.37.2 log_callback
1.1.37.3 EBuff_Type
1.1.37.4 ETask_Msg_Type
1.1.37.5 PCIe Buffer Node
1.1.37.6 PCIe Device Bus
1.1.37.7 PCIe EP Device Mode
1.1.37.8 pcie_dev_attr_st
1.1.37.9 PCIe Task Attribute Structure
1.1.37.10 pcie_task_msg_st
1.1.37.11 pcie_ep_obj_info_for_user

1.1.37.12 pcie_err_event_count
1.1.37.13 pcie_replace_buff_info_st
1.1.37.14 pcie_replace_buff_info_list_st
1.2 PCIE Windows EP
1.2.1 Data Types
1.2.1.1 RK_PCIE_TASK_ATTR
1.2.1.2 RK_PCIE_DESTROY_TASK
1.2.1.3 RK_PCIE_IOCTL_STATUS
1.2.1.4 RK_PCIE_TASK_MSG
1.2.1.5 RK_PCIE_BUFF_NODE
1.2.1.6 RK_PCIE_DEV_MODE
1.2.1.7 RK_PCIE_TASK_INFO
1.2.1.8 RK_PCIE_DOWNLOAD_FIRMWARE
1.3 PCIE Endpoint
1.3.1 rk_pcie_set_log_level
1.3.2 rk_pcie_set_log_cb
1.3.3 rk_pcie_device_init
1.3.4 rk_pcie_device_deinit
1.3.5 rk_pcie_task_create
1.3.6 rk_pcie_task_destroy
1.3.7 rk_pcie_set_ep_info
1.3.8 rk_pcie_get_user_cmd
1.3.9 rk_pcie_get_bar1_user_buffer
1.3.10 rk_pcie_get_bar5_user_buffer
1.3.11 rk_pcie_set_bar_inbound_cpu_addr
1.3.12 rk_pcie_get_buff
1.3.13 rk_pcie_send_buff
1.3.14 rk_pcie_release_buff
1.3.15 rk_pcie_get_buff_max_size
1.3.16 rk_pcie_get_msg_for_bus
1.3.17 rk_pcie_send_msg_to_rc_task
1.3.18 rk_pcie_check_buff_avail
1.3.19 rk_pcie_get_err_event_count
1.3.20 rk_pcie_is_alive
1.3.21 rk_pcie_replace_buff_all
1.3.22 rk_pcie_replace_buff
1.3.23 Data Types
1.3.23.1 EBuff_Type
1.3.23.2 ETask_Msg_Type
1.3.23.3 PCIe Buffer Node
1.3.23.4 pcie_task_msg_st
1.4 Error Codes

# 1. Chips Connect API Reference

## 1.1 PCIe Linux Root Complex

### 1.1.1 rk_pcie_get_pci_bus_addresses

[Description]

Retrieve all matching PCI device bus addresses in the system.

[Syntax]

```
int rk_pcie_get_pci_bus_addresses(int vendor_id, int device_id, struct pcie_dev_bus
*pci_bus)
```

[Parameters]

| Parameter Name | Description | Input/Output |
| --- | --- | --- |
| vendor_id | PCIe Vendor Identifier | Input |
| device_id | PCIe Product Identifier | Input |
| pci_bus | PCI device bus address struct | Output |

[Return Values]

| Return Value | Description |
| --- | --- |
| 0 | Success |
| Non-zero | Failure, please refer to Error Codes. |

[Example]

Please refer to the example in rk_pcie_device_init.

### 1.1.2 rk_pcie_device_init

[Description]

Initialize the PCIe device.

[Syntax]

```
RK_PCIE_HANDLES rk_pcie_device_init(struct pcie_dev_attr_st *dev)
```

[Parameters]

| Parameter Name | Description | Input/Output |
| --- | --- | --- |
| dev | PCIe device configuration structure | Input |

[Return Values]

| Return Value | Description |
| --- | --- |
| Non-NULL | Success |
| NULL | Failure |

[Example]

```
#define VENDOR_ID                   0x1d87
#define DEVICE_ID                   0x356a

struct pcie_dev_bus dev_bus;
struct pcie_dev_attr_st dev_attr;
RK_PCIE_HANDLES rk_handle;
int ret;

ret = rk_pcie_get_pci_bus_addresses(VENDOR_ID, DEVICE_ID, &dev_bus);
if (ret != 0) {
    printf("get pci bus address fail\n");
    return -1;
}

printf("ep dev max num : %d\n", dev_bus.dev_max_num);
if (dev_bus.dev_max_num == 0) {
    printf("no ep device, exit ... \n");
} else {
    // Initialize PCIe device
    dev_attr.using_rc_dma = 0; // Disable RC DMA, use EP DMA transfer
    dev_attr.rc_dma_chn = 0;
    dev_attr.vendor_id = VENDOR_ID;
    dev_attr.device_id = DEVICE_ID;
    dev_attr.use_dma_only_in_rc = 0;
    // Select the first matching device
    memcpy(dev_attr.pci_bus_address, dev_bus.pci_bus_address[0],
strlen(dev_bus.pci_bus_address[0]));

    // Initialize EP device
    rk_handle = rk_pcie_device_init(&dev_attr);
    if (rk_handle == NULL) {
        printf("pcie device init fail:%s \n", dev_attr.pci_bus_address);
        return -1;
```

```
    }

    // Deinitialize PCIe device, release resources
    ret = rk_pcie_device_deinit(rk_handle, 0);
    if (ret != 0) {
        printf("pcie deinit device fail....ret = %d\n", ret);
        return -1;
    }
}
```

[Related Topics]

rk_pcie_device_deinit

### 1.1.3 rk_pcie_device_deinit

[Description]

Deinitialize the PCIe device and release resources.

[Syntax]

```
int rk_pcie_device_deinit(RK_PCIE_HANDLES handles, int en_remove)
```

[Parameters]

| Parameter Name | Description | Input/Output |
| --- | --- | --- |
| handles | Device handle | Input |
| en_remove | Release resources and remove the device | Input |

[Return Values]

| Return Value | Description |
| --- | --- |
| 0 | Success |
| -1 | Invalid handle |
| -2 | Device not initialized |
| Others | Exception, please refer to Error Codes |

[Example]

Please refer to the example in rk_pcie_device_init.

[Related Topics]

rk_pcie_device_init

### 1.1.4 rk_pcie_task_create

[Description]

Create a task based on attributes, allocate corresponding memory, and manage BUFFs. This interface will block and wait for the EP device to call the task creation interface.

[Syntax]

```
int rk_pcie_task_create(RK_PCIE_HANDLES handles, struct pcie_task_attr_st *task)
```

[Parameters]

| Parameter Name | Description | Input/Output |
| --- | --- | --- |
| handles | Device handle | Input |
| task | PCIe Task configuration structure | Input |

[Return Values]

| Return Value | Description |
| --- | --- |
| Greater than 0 | Normal, task_id |
| Less than 0 | Exception, please refer to Error Codes |

[Example]

```
// Select the first matching device
rc_ctx.rk_handle = rk_pcie_device_init(&rc_ctx.dev_attr);
if (rc_ctx.rk_handle == NULL) {
    printf("pcie device init fail:%s \n", rc_ctx.dev_attr.pci_bus_address);
    return -1;
}

rc_ctx.task_attr.send_buff_num = 5; // Number of send buffers
rc_ctx.task_attr.send_buff_size = 1048576;   // Size of send buffer
rc_ctx.task_attr.recv_buff_num = 5; // Number of receive buffers
rc_ctx.task_attr.recv_buff_size = 8294408 ;  // Size of receive buffer
rc_ctx.task_attr.wait_timeout_ms = -1;  // Blocking wait
rc_ctx.task_id = rk_pcie_task_create(rc_ctx.rk_handle, &rc_ctx.task_attr);
if (rc_ctx.task_id < 0) {
    printf("pcie init create task fail....\n");
    return -1;
}

rk_pcie_get_ep_mode(rc_ctx.rk_handle, &rc_ctx.devmode);
printf("ep ap init, dev mode: %d, submode: %d\n", rc_ctx.devmode.mode,
rc_ctx.devmode.submode);
```

```
ret = rk_pcie_task_destroy(rc_ctx.rk_handle, rc_ctx.task_id);
if (ret != 0) {
    printf("pcie destroy task fail....ret = %d\n", ret);
    return -1;
}
```

[Related Topics]

rk_pcie_task_destroy

[Note]

This interface depends on `rk_pcie_device_init`, and `rk_pcie_device_init` must be called first to initialize the device before task initialization.

### 1.1.5 rk_pcie_task_destroy

[Description]

Deinitialize the Task, releasing the memory and BUFF management resources associated with the Task.

[Syntax]

```
int rk_pcie_task_destroy(RK_PCIE_HANDLES handles, int task_id)
```

[Parameters]

| Parameter Name | Description | Input/Output |
| --- | --- | --- |
| handles | Device handle | Input |
| task_id | Task ID | Input |

[Return Values]

| Return Value | Description |
| --- | --- |
| 0 | Normal |
| Non-zero | Failure, please refer to Error Codes. |

[Example]

Please refer to the example in rk_pcie_task_create.

[Related Topics]

rk_pcie_task_create

### 1.1.6 rk_pcie_boot_init

[Description]

Initializes PCIE BOOT.

[Syntax]

```
int rk_pcie_boot_init(RK_PCIE_HANDLES handles)
```

[Parameters]

| Parameter Name | Description | Input/Output |
| --- | --- | --- |
| handles | Device handle | Input |

[Return Values]

| Return Value | Description |
| --- | --- |
| 0 | Normal |
| Non-zero | Failure, see Error Codes. |

[Example]

```
// Initialize PCIE device
rc_ctx.rk_handle = rk_pcie_device_init(&rc_ctx.dev_attr);
if (rc_ctx.rk_handle == NULL) {
    printf("device init fail \n");
    return -1;
}

// Initialize PCIE Boot
ret = rk_pcie_boot_init(rc_ctx.rk_handle);
if (ret != 0) {
    printf("pcie boot init ep fail....ret = %d\n", ret);
    return -1;
}

// pcie boot
rk_pcie_boot_download_firmware_list(&rc_ctx, &pcie_boot_info, &pcie_boot_ini);

// ep boot
rk_ep_boot_download_firmware_list(&rc_ctx, &ep_boot_info, &ep_boot_ini);

// Release PCIE Boot resources
ret = rk_pcie_boot_deinit(rc_ctx.rk_handle);
if (ret != 0) {
    printf("pcie boot deinit ep fail....ret = %d\n", ret);
    return -1;
}

printf("download ep end\n");
```

[Related Topics]

rk_pcie_boot_deinit

[Note]

This interface depends on `rk_pcie_device_init`, you must call `rk_pcie_device_init` to initialize the device before proceeding with boot initialization.

### 1.1.7 rk_pcie_boot_deinit

[Description]

Deinitializes the PCIe BOOT.

[Syntax]

```
int rk_pcie_boot_deinit(RK_PCIE_HANDLES handles)
```

[Parameters]

| Parameter Name | Description | Input/Output |
| --- | --- | --- |
| handles | Device handle | Input |

[Return Values]

| Return Value | Description |
| --- | --- |
| 0 | Normal |
| Non-zero | Failure, please refer to Error Codes. |

[Example]

rk_pcie_boot_init

[Related Topics]

rk_pcie_boot_init

### 1.1.8 rk_pcie_boot_download_firmware

[Description]

Download the PCIE BOOT firmware.

[Syntax]

```
int rk_pcie_boot_download_firmware(RK_PCIE_HANDLES handles, unsigned char *firmware,
unsigned long long firmware_size, unsigned long long remote_load_addr)
```

[Parameters]

| Parameter Name | Description | Input/Output |
| --- | --- | --- |
| handles | Device handle | Input |
| firmware | Firmware data | Input |
| firmware_size | Size of firmware | Input |
| remote_load_addr | Remote firmware load address | Input |

[Return Values]

| Return Value | Description |
| --- | --- |
| 0 | Normal |
| Non-zero | Failure, please refer to Error Codes. |

[Example]

```
int rk_ep_download_firmware(struct rc_context_st *rc_ctx, char *firmware, unsigned long
load_addr)
{
    int fd;
    unsigned long len;
    unsigned long size = 1024 * 1024;
    void *buffer = malloc(size);
    int ret;
    fd = open(firmware, O_RDONLY);
    if (fd == -1) {
        printf(" open %s file error, ret = %d \n", firmware, fd);
        return -1;
    }

    while ((len = read(fd, buffer, size)) > 0) {
        ret = rk_pcie_boot_download_firmware(rc_ctx->rk_handle, buffer, len, load_addr);
        if (ret != 0) {
            return -1;
        }

        load_addr += len;
    }

    close(fd);
    return 0;
}
```

[Related Topics]

rk_ep_download_firmware

### 1.1.9 rk_pcie_boot_run

[Description]

Boot the PCIe BOOT firmware.

[Syntax]

```
int rk_pcie_boot_run(RK_PCIE_HANDLES handles)
```

[Parameters]

| Parameter Name | Description | Input/Output |
| --- | --- | --- |
| handles | Device handle | Input |

[Return Values]

| Return Value | Description |
| --- | --- |
| 0 | Normal |
| Non-zero | Failure, please refer to Error Codes. |

[Example]

rk_pcie_boot_init

### 1.1.10 rk_pcie_get_ep_info

[Description]

Retrieve EP status information, with a maximum support of 96KB.

[Syntax]

```
int rk_pcie_get_ep_info(RK_PCIE_HANDLES handles, void *status, size_t status_size)
```

[Parameters]

| Parameter Name | Description | Input/Output |
| --- | --- | --- |
| handles | Device handle | Input |
| status | void pointer, status structure defined by the user | Output |
| status_size | Size of the status structure | Input |

[Return Values]

| Return Value | Description |
| --- | --- |
| 0 | Normal |
| Non-zero | Failure, please refer to Error Codes. |

[Example]

```
struct ep_info_st {
    unsigned int link_status;
    unsigned int temp;
    long long timestamp;
    unsigned char cpu_load;
    unsigned char npu_load;
    unsigned char gpu_load;
};

long long timestamp;
while(1) {
    ret = rk_pcie_get_ep_info(rc_ctx.rk_handle, ep_info, sizeof(struct ep_info_st));
    if(timestamp == ep_info->timestamp) {
        printf("EP device status is abnormal \n");
        break;
    }
    timestamp = ep_info->timestamp;
}
```

### 1.1.11 rk_pcie_get_user_cmd

[Description]

Obtain a memory pointer at a specific location of BAR0 for user-defined command areas. Supports up to 256KB.

[Syntax]

```
void *rk_pcie_get_user_cmd(RK_PCIE_HANDLES handles)
```

[Parameters]

| Parameter Name | Description | Input/Output |
| --- | --- | --- |
| handles | Device handle | Input |

[Return Value]

| Return Value | Description |
| --- | --- |
| Non-NULL | Normal, pointer to custom command area |
| NULL | Failure |

[Example]

```
struct user_cmd_st {
    unsigned int max_chn;
    unsigned int width;
    unsigned int height;
    unsigned int codecid;
    unsigned int output_id;
    unsigned int init;
};

user_cmd = (struct user_cmd_st *)rk_pcie_get_user_cmd(rc_ctx.rk_handle);
user_cmd->max_chn = rc_ctx.max_chn;
user_cmd->width = rc_ctx.width;
user_cmd->height = rc_ctx.height;
user_cmd->codecid = rc_ctx.codecid;
user_cmd->output_id = 1; //0:RGB888 1:BGRA8888
user_cmd->init = 1;
```

### 1.1.12 rk_pcie_get_buff

[Description]

Obtain an available PCIe buffer.

[Syntax]

```
struct pcie_buff_node *rk_pcie_get_buff(RK_PCIE_HANDLES handles, int task_id, enum EBuff_Type type)
```

[Parameters]

| Parameter Name | Description | Input/Output |
| --- | --- | --- |
| handles | Device handle | Input |
| task_id | Task ID | Input |
| type | PCIe Buff Type | Input |

[Return Values]

| Return Value | Description |
| --- | --- |
| Non-NULL | Pointer to available Buff |
| NULL | No available Buff pointer |

[Example]

```
node = rk_pcie_get_buff(ctx->rk_handle, ctx->task_id, E_RECV); //Obtain an available receive
BUF
if (node) {
    if (check_info(node->vir_addr + node->size - CHECK_SIZE) == 0) { //Verify the BUF data is
correct
        pthread_mutex_lock(&ctx->mutex_stream_speed);
        ctx->stream_speed_index++;
        pthread_mutex_unlock(&ctx->mutex_stream_speed);
        memset(node->vir_addr + node->size - CHECK_SIZE, 0, CHECK_SIZE);
    } else {
        printf("check data ==================== error \n");
        for (int i = 0; i < CHECK_SIZE; i++) {
            unsigned char *p = node->vir_addr + node->size - CHECK_SIZE;
            printf("%d ", p[i]);
        }
        printf("\ncheck data ===ssss========node->size = %d======== error \n", (int)node-
>size);
    }
    rk_pcie_release_buff(ctx->rk_handle, ctx->task_id, node);  //Release BUF
} else {
    usleep(1000);
}
```

[Related Topics]

rk_pcie_release_buff

### 1.1.13 rk_pcie_send_buff

[Description]

Sends the PCIE buffer, and blocks until DMA transfer is completed.

[Syntax]

```
int rk_pcie_send_buff(RK_PCIE_HANDLES handles, int task_id, struct pcie_buff_node *pbuff)
```

[Parameters]

| Parameter Name | Description | Input/Output |
| --- | --- | --- |
| handles | Device handle | Input |
| task_id | Task ID | Input |
| pbuff | PCIE Buffer pointer | Input |

[Return Value]

| Return Value | Description |
| --- | --- |
| 0 | Send successful, indicating that the data has been transferred via DMA. |
| Non-zero | Failure, please refer to Error Codes. |

[Example]

```
node = rk_pcie_get_buff(ctx->rk_handle, ctx->task_id, E_SEND); // Get an available send
buffer
if (node) {
    node->size = buffer_size;
    // Operate on the buffer, write data
    memcpy(node->vir_addr, mptr, node->size);
    add_check_info(node->vir_addr + node->size - CHECK_SIZE);
    rk_pcie_send_buff(ctx->rk_handle, ctx->task_id, node); // Send to the other end
    index++;
} else {
    usleep(1000);
}
```

### 1.1.14 rk_pcie_get_bar1_user_buffer

[Description]

Obtaining the virtual mapping address of bar1.

[Syntax]

```
void *rk_pcie_get_bar1_user_buffer(RK_PCIE_HANDLES handles)
```

[Parameters]

| Parameter Name | Description | Input/Output |
| --- | --- | --- |
| handles | Handle of all matching devices | Input |

[Return Values]

| Return Value | Description |
| --- | --- |
| void * | Virtual address |
| NULL | Failure, see Error Codes. |

### 1.1.15 rk_pcie_get_bar5_user_buffer

[Description]

Obtains the virtual mapping address of bar5.

[Syntax]

```
void *rk_pcie_get_bar5_user_buffer(RK_PCIE_HANDLES handles)
```

[Parameters]

| Parameter Name | Description | Input/Output |
| --- | --- | --- |
| handles | Handle of all matching devices | Input |

[Return Values]

| Return Value | Description |
| --- | --- |
| void * | Virtual address |
| NULL | Failure, see Error Codes. |

### 1.1.16 rk_pcie_set_bar_inbound_cpu_addr

[Description]

Set BAR inbound mapping.

[Syntax]

```
u_int64_t rk_pcie_set_bar_inbound_cpu_addr(RK_PCIE_HANDLES handles, u_int32_t atu_index,
u_int64_t cpu_address, u_int32_t size)
```

[Parameters]

| Parameter Name | Description | Input/Output |
| --- | --- | --- |
| handles | Handle of all matching devices | Input |
| atu_index | ATU index | Input |
| cpu_address | Mapped CPU address | Input |
| size | Mapping size limit | Input |

[Return Values]

| Return Value | Description |
| --- | --- |
| cpu_address | Returns CPU address |

### 1.1.17 rk_pcie_release_buff

[Description]

Release the PCIE buffer.

[Syntax]

```
int rk_pcie_release_buff(RK_PCIE_HANDLES handles, int task_id, struct pcie_buff_node *pbuff)
```

[Parameters]

| Parameter Name | Description | Input/Output |
| --- | --- | --- |
| handles | Handle for all matching devices | Input |
| task_id | Task ID | Input |
| pbuff | Pointer to PCIE Buffer | Input |

[Return Values]

| Return Value | Description |
| --- | --- |
| 0 | Successfully released |
| Non-zero | Failure, see Error Codes. |

[Example]

rk_pcie_get_buff

### 1.1.18 rk_pcie_get_buff_max_size

[Description]

Obtaining the maximum size supported by the buffer.

[Syntax]

```
long rk_pcie_get_buff_max_size(RK_PCIE_HANDLES handles, int task_id, enum EBuff_Type type)
```

[Parameters]

| Parameter Name | Description | Input/Output |
| --- | --- | --- |
| handles | Handle of all matching devices | Input |
| task_id | Task ID | Input |
| type | Type of PCIE Buffer | Input |

[Return Values]

| Return Value | Description |
| --- | --- |
| Greater than or equal to 0 | Buffer size |
| Less than 0 | Failure, see Error Codes |

[Example]

```
long buffer_size = rk_pcie_get_buff_max_size(ctx->rk_handle, ctx->task_id, E_SEND);
```

### 1.1.19 rk_pcie_get_ep_mode

[Description]

Retrieve the current operating mode of the EP device, supporting status acquisition from loader, kernel, application, and other modes. Mode information can be obtained in rk_pcie_rc.h

[Syntax]

```
int rk_pcie_get_ep_mode(RK_PCIE_HANDLES handles, struct pcie_dev_mode *devmode)
```

[Parameters]

| Parameter Name | Description | Input/Output |
| --- | --- | --- |
| handles | Device handle | Input |
| devmode | Device mode structure | Output |

[Return Values]

| Return Value | Description |
| --- | --- |
| 0 | Success |
| Non-zero | Failure, please refer to Error Codes |

[Example]

```
struct rc_context_st {
    RK_PCIE_HANDLES rk_handle;
    struct pcie_task_attr_st dev_attr;
    struct pcie_dev_mode devmode;
}rc_ctx;

// Initialize the first matching device
rc_ctx.rk_handle = rk_pcie_device_init(&rc_ctx.dev_attr);
if (rc_ctx.rk_handle == NULL) {
    printf("device init fail \n");
    usleep(1000000);
    continue;
}

rk_pcie_get_ep_mode(rc_ctx.rk_handle, &rc_ctx.devmode);
printf("111 ep device init, dev mode: 0x%.2x, submode: 0x%.2x\n", rc_ctx.devmode.mode,
rc_ctx.devmode.submode);
```

### 1.1.20 Rescanning PCI Devices

[Description]

Rescanning system PCI devices is necessary for EP devices to recover after an abnormal restart by removing and then rescanning the devices.

[Syntax]

```
void rk_pcie_rescan_devices(void)
```

[Parameters]

None

[Return Value]

None

[Example]

None

### 1.1.21 rk_pcie_send_msg_to_ep_bus

[Description]

Send a message to the EP side, with a maximum of 1504Bytes per message.

[Syntax]

```
int rk_pcie_send_msg_to_ep_bus(RK_PCIE_HANDLES handles, int task_id, void *data, size_t size)
```

[Parameters]

| Parameter Name | Description | Input/Output |
| --- | --- | --- |
| handles | Device handle | Input |
| task_id | Task ID | Input |
| data | Message pointer | Input |
| size | Message length | Input |

[Return Values]

| Return Value | Description |
| --- | --- |
| 0 | Success |
| Non-zero | Failure, please refer to Error Codes |

[Example]

```
rc_ctx.task_attr.send_buff_num = 5;
rc_ctx.task_attr.send_buff_size = 5 * 1024 * 1024;
rc_ctx.task_attr.recv_buff_num = 5;
rc_ctx.task_attr.recv_buff_size = 5 * 1024 * 1024;
rc_ctx.task_attr.wait_timeout_ms = -1;
rc_ctx.task_id = rk_pcie_task_create(rc_ctx.rk_handle, &rc_ctx.task_attr);
if (rc_ctx.task_id < 0) {
    printf("pcie init create task fail....\n");
    return -1;
}

rk_pcie_get_ep_mode(rc_ctx.rk_handle, &rc_ctx.devmode);
printf("ep ap init, dev mode: 0x%.2x, submode: 0x%.2x\n", rc_ctx.devmode.mode,
rc_ctx.devmode.submode);

rc_ctx.user_cmd.cmd = EP_TEST_STATUS_RUN_APP;
rk_pcie_send_msg_to_ep_bus(rc_ctx.rk_handle, rc_ctx.task_id, (void *)&rc_ctx.user_cmd,
sizeof(struct task_user_cmd_st));
```

### 1.1.22 rk_pcie_get_msg_for_task_id

[Description]

Retrieve EP messages through Task ID

[Syntax]

```
int rk_pcie_get_msg_for_task_id(RK_PCIE_HANDLES handles, struct pcie_task_msg_st *msg, int
task_id, int timeout_ms)
```

[Parameters]

| Parameter Name | Description | Input/Output |
| --- | --- | --- |
| handles | Device handle | Input |
| msg | Pointer to message structure | Output |
| task_id | Task ID | Input |
| timeout_ms | Timeout in milliseconds, -1 for blocking mode (not recommended, will wait in kernel indefinitely), recommended 100ms | Input |

[Return Values]

| Return Value | Description |
| --- | --- |
| 0 | Success |
| Non-zero | Failure, please refer to Error Codes |

[Example]

```
unsigned char cmd;
struct pcie_task_msg_st msg;
ret = rk_pcie_get_msg_for_task_id(rc_ctx.rk_handle, &msg, rc_ctx.task_id, 100);
if (ret == RK_PCIE_OK) {
    cmd = msg.data[0];
    printf("recv ep msg: %d \n", cmd);
    if (cmd == 10) {
        break;
    }
}
```

### 1.1.23 rk_pcie_get_ep_obj_info

[Description]

Retrieve basic information of the EP device

[Syntax]

```
int rk_pcie_get_ep_obj_info(RK_PCIE_HANDLES handles, struct pcie_ep_obj_info_for_user *info)
```

[Parameters]

| Parameter Name | Description | Input/Output |
| --- | --- | --- |
| handles | Device handle | Input |
| info | Buffer for basic info | Output |

[Return Values]

| Return Value | Description |
| --- | --- |
| 0 | Success |
| Non-zero | Failure, see Error Codes |

[Example]

```
struct pcie_ep_obj_info_for_user info;
ret = rk_pcie_get_ep_obj_info(rc_ctx.rk_handle, &info);
if (ret == RK_PCIE_OK) {
    return ret;
}
```

### 1.1.24 rk_pcie_get_ep_err_event_count

[Description]

Get the PCIe EP error event count.

[Syntax]

```
int rk_pcie_get_ep_err_event_count(RK_PCIE_HANDLES handles, struct pcie_err_event_count
*info)
```

[Parameters]

| Parameter Name | Description | Input/Output |
| --- | --- | --- |
| handles | Device handle | Input |
| info | PCIe error event count | Output |

[Return Values]

| Return Value | Description |
| --- | --- |
| 0 | Success |
| Non-zero | Failure, refer to Error Codes |

[Example]

```
struct pcie_err_event_count info;
ret = rk_pcie_get_ep_err_event_count(rc_ctx.rk_handle, &info);
if (ret == RK_PCIE_OK) {
    return ret;
}
```

### 1.1.25 rk_pcie_is_alive

[Description]

Check if the PCIe link is normal.

[Syntax]

```
int rk_pcie_is_alive(RK_PCIE_HANDLES handles);
```

[Parameters]

| Parameter Name | Description | Input/Output |
| --- | --- | --- |
| handles | Device handle | Input |

[Return Values]

| Return Value | Description |
| --- | --- |
| cc_true | Link is normal |
| cc_false | Link is abnormal |

[Example]

```
ret = rk_pcie_is_alive(rc_ctx.rk_handle);
if (!ret) {
    printf("PCIe link down\n");
    return ret;
}
```

### 1.1.26 rk_pcie_device_hardware_reset

[Description]

Custom kernel driver interface for hardware reset controller. The device driver needs to implement the PCIE_EP_RESET_CTRL ioctl operation. During the process, the PERST# signal will be controlled to hardware reset the EP.

[Syntax]

```
int rk_pcie_device_hardware_reset(RK_PCIE_HANDLES handles);
```

[Parameters]

| Parameter Name | Description | Input/Output |
| --- | --- | --- |
| handles | Device handle | Input |

[Return Values]

| Return Value | Description |
| --- | --- |
| 0 | Reset successful |
| Non-zero | Reset failed or not supported, refer to Error Codes |

[Example]

Before resetting the controller, stop all PCIe transactions. For example:

```
printf("Reset beginning...\n");
/* Reset Progress: Stop all transmission */
rc_ctx.recv_ep_exit = 1;
rc_ctx.send_ep_exit = 1;
for (int i = 0; i < THREADS_MAX; i++) {
    pthread_join(rc_ctx.thr_recv_ep[i], NULL);
    pthread_join(rc_ctx.thr_send_ep[i], NULL);
}
usleep(500000);

/* Reset Progress: Destroy task */
ret = rk_pcie_task_destroy(rc_ctx.rk_handle, rc_ctx.task_id);
if (ret != 0) {
    printf("pcie destroy task fail....ret = %d\n", ret);
    return -1;
}
usleep(100000);

/* Reset Progress: Hardware reset */
ret = rk_pcie_device_hardware_reset(rc_ctx.rk_handle);
if (ret != 0) {
    printf("pcie device hardware reset fail....ret = %d\n", ret);
}
```

```
/* Reset Progress: Wait for link up */
usleep(1000000);

/* Reset Progress: Deinit device */
ret = rk_pcie_device_deinit(rc_ctx.rk_handle, 1);
if (ret != 0) {
    printf("pcie deinit device fail....ret = %d\n", ret);
    return -1;
}

/* Reset Progress: Rescan PCI */
rk_pcie_rescan_devices();
printf("Reset done\n");

return 0;
```

### 1.1.27 rk_pcie_set_log_level

[Description]

Set the log print level, default is debug.

[Syntax]

```
int rk_pcie_set_log_level(int level)
```

[Parameters]

| Parameter Name | Description | Input/Output |
| --- | --- | --- |
| level | Log level, options: 6(verbose), 5(debug), 4(info), 3(warn), 2(error), 1(fatal) | Input |

[Return Values]

| Return Value | Description |
| --- | --- |
| 0 | Success |
| Non-zero | Failure, please refer to Error Codes |

### 1.1.28 rk_pcie_set_log_cb

[Description]

Set log callback function to redirect logs from default console output to caller.

[Syntax]

```
int rk_pcie_set_log_cb(log_callback cb)
```

[Parameters]

| Parameter Name | Description | Input/Output |
| --- | --- | --- |
| cb | Log callback function pointer, type is `int (*log_callback)(char *log)` | Input |

[Return Values]

| Return Value | Description |
| --- | --- |
| 0 | Success |
| Non-zero | Failure, please refer to Error Codes |

### 1.1.29 rk_pcie_device_hardware_reset_sync

[Description]

Custom kernel driver interface for hardware reset controller with synchronization. Waits for EP confirmation before reset. During the process, the PERST# signal will be controlled to hardware reset the EP.

[Syntax]

```
int rk_pcie_device_hardware_reset_sync(RK_PCIE_HANDLES handles, int task_id, int timeout_ms);
```

[Parameters]

| Parameter Name | Description | Input/Output |
| --- | --- | --- |
| handles | Device handle | Input |
| task_id | Task ID | Input |
| timeout_ms | Timeout in milliseconds | Input |

[Return Values]

| Return Value | Description |
| --- | --- |
| 0 | Reset successful |
| Non-zero | Reset failed or not supported, refer to Error Codes |

### 1.1.30 rk_pcie_is_device_hardware_reset

[Description]

Check if PCIe link is currently in reset state.

[Syntax]

```
int rk_pcie_is_device_hardware_reset(RK_PCIE_HANDLES handles);
```

[Parameters]

| Parameter Name | Description | Input/Output |
| --- | --- | --- |
| handles | Device handle | Input |

[Return Values]

| Return Value | Description |
| --- | --- |
| cc_true | Reset in progress |
| cc_false | Not in reset state |

### 1.1.31 rk_pcie_boot_upload_firmware

[Description]

Upload firmware data from EP side.

[Syntax]

```
int rk_pcie_boot_upload_firmware(RK_PCIE_HANDLES handles, unsigned char *firmware, unsigned long long firmware_size, unsigned long long remote_load_addr)
```

[Parameters]

| Parameter Name | Description | Input/Output |
| --- | --- | --- |
| handles | Device handle | Input |
| firmware | Firmware data buffer | Output |
| firmware_size | Firmware size | Input |
| remote_load_addr | Remote firmware read address | Input |

[Return Values]

| Return Value | Description |
| --- | --- |
| 0 | Success |
| Non-zero | Failure, please refer to Error Codes |

### 1.1.32 rk_pcie_get_boot_state

[Description]

Get EP boot state.

[Syntax]

```
int rk_pcie_get_boot_state(RK_PCIE_HANDLES handles, unsigned int off)
```

[Parameters]

| Parameter Name | Description | Input/Output |
| --- | --- | --- |
| handles | Device handle | Input |
| off | Status register offset | Input |

[Return Values]

| Return Value | Description |
| --- | --- |
| >=0 | Boot state value |
| <0 | Failure, please refer to error codes |

### 1.1.33 rk_pcie_set_fw_info

[Description]

Set firmware information to EP side.

[Syntax]

```
int rk_pcie_set_fw_info(RK_PCIE_HANDLES handles, unsigned char *buf, unsigned int off,
unsigned int size)
```

[Parameters]

| Parameter Name | Description | Input/Output |
| --- | --- | --- |
| handles | Device handle | Input |
| buf | Firmware info data | Input |
| off | Write offset | Input |
| size | Data size | Input |

[Return Values]

| Return Value | Description |
| --- | --- |
| 0 | Success |
| Non-zero | Failure, please refer to Error Codes |

### 1.1.34 rk_pcie_send_merge_msg_to_ep_bus

[Description]

Send merged message to EP side for data that needs to be aggregated.

[Syntax]

```
int rk_pcie_send_merge_msg_to_ep_bus(RK_PCIE_HANDLES handles, int task_id, void *data,
unsigned long long size)
```

[Parameters]

| Parameter Name | Description | Input/Output |
| --- | --- | --- |
| handles | Device handle | Input |
| task_id | Task ID | Input |
| data | Message pointer | Input |
| size | Message length | Input |

[Return Values]

| Return Value | Description |
| --- | --- |
| 0 | Success |
| Non-zero | Failure, please refer to Error Codes |

### 1.1.35 rk_pcie_replace_buff_all

[Description]

Replace all buff addresses for configuring user-defined buffs.

[Syntax]

```
int rk_pcie_replace_buff_all(RK_PCIE_HANDLES handle, int task_id, enum EBuff_Type type,
struct pcie_replace_buff_info_list_st *infos)
```

[Parameters]

| Parameter Name | Description | Input/Output |
| --- | --- | --- |
| handle | Device handle | Input |
| task_id | Task ID | Input |
| type | Buff type (E_SEND or E_RECV) | Input |
| infos | Replace buff info list | Input |

[Return Values]

| Return Value | Description |
| --- | --- |
| 0 | Success |
| Non-zero | Failure, please refer to Error Codes |

[Related Topics]

rk_pcie_replace_buff pcie_replace_buff_info_list_st

## 1.1.36 rk_pcie_replace_buff

[Description]

Replace single buff address for configuring user-defined buffs.

[Syntax]

```
int rk_pcie_replace_buff(RK_PCIE_HANDLES handle, int task_id, enum EBuff_Type type, struct
pcie_buff_node *pbuff, pcie_replace_buff_info_st *info)
```

[Parameters]

| Parameter Name | Description | Input/Output |
| --- | --- | --- |
| handle | Device handle | Input |
| task_id | Task ID | Input |
| type | Buff type (E_SEND or E_RECV) | Input |
| pbuff | Original buff node pointer | Input |
| info | New buff info | Input |

[Return Values]

| Return Value | Description |
| --- | --- |
| 0 | Success |
| Non-zero | Failure, please refer to Error Codes |

[Related Topics]

rk_pcie_replace_buff_all pcie_replace_buff_info_st

## 1.1.37 Data Types

PCIE RC-related data types and data structure definitions are as follows:

- RK_PCIE_HANDLES: Defines the device handle.
- log_callback: Defines the log callback function type.
- EBuff_Type: Defines the PCIE Buff type.
- ETask_Msg_Type: Defines the Task message type.
- struct pcie_buff_node: Defines the PCIE buff attribute structure.
- struct pcie_dev_bus: Defines the PCI device bus information.
- struct pcie_dev_mode: Defines the PCIE EP device mode.
- struct pcie_dev_attr_st: Defines the PCIE EP device attribute structure.
- struct pcie_task_attr_st: Defines the Task attribute structure.
- struct pcie_task_msg_st: Defines the Task message structure.
- struct pcie_ep_obj_info_for_user: Defines the PCIe basic information structure.
- struct pcie_err_event_count: Defines the PCIe error event information structure.
- pcie_replace_buff_info_st: Defines the replace buff info structure.
- struct pcie_replace_buff_info_list_st: Defines the replace buff info list structure.

### 1.1.37.1 RK_PCIE_HANDLES

[Description]

Defines the device handle.

[Definition]

```
typedef void* RK_PCIE_HANDLES;
```

#### 1.1.37.2 log_callback

[Description]

Defines the log callback function type.

[Definition]

```
typedef int (*log_callback)(char *log);
```

[Parameters]

| Parameter Name | Description |
| --- | --- |
| log | Log string passed by the caller |

#### 1.1.37.3 EBuff_Type

[Description]

Defines the types of PCIE Buff.

[Definition]

```
enum EBuff_Type {
    E_SEND = 0,
    E_RECV,
};
```

[Members]

| Member Name | Description |
| --- | --- |
| E_SEND | Sending buffer. |
| E_RECV | Receiving buffer. |

#### 1.1.37.4 ETask_Msg_Type

[Explanation]

Defines the types of PCIE messages.

[Definition]

```
enum ETask_Msg_Type {
    E_TASK_MSG_NONE = 0,
    E_TASK_MSG_CREATE,
    E_TASK_MSG_DESTROY,
    E_TASK_MSG_USER,
    E_TASK_MSG_MAX,
};
```

[Members]

| Member Name | Description |
| --- | --- |
| E_TASK_MSG_NONE | Default value, no significance. |
| E_TASK_MSG_CREATE | Task creation. |
| E_TASK_MSG_DESTROY | Task destruction. |
| E_TASK_MSG_USER | User-defined message. |
| E_TASK_MSG_MAX | Maximum message type. |

#### 1.1.37.5 PCIe Buffer Node

[Explanation]

Defines the PCIE buffer attribute structure.

[Definition]

```
struct pcie_buff_node{
    unsigned char *vir_addr;
    unsigned long long phy_addr;
    unsigned int size;
};
```

[Members]

| Member Name | Description |
| --- | --- |
| vir_addr | Virtual address. |
| phy_addr | Physical address. |
| size | Memory size.<br>Sending buffer: Users need to configure the data size, which should not exceed the total buffer memory size.<br>Receiving buffer: The size of the data sent by the other end. |

#### 1.1.37.6 PCIe Device Bus

[Explanation]

Defines the bus information for PCI devices.

[Definition]

```
struct pcie_dev_bus {
    int dev_max_num;
    char pci_bus_address[PCI_DEV_MAX][PCI_BUS_MAX_NAME];
};
```

[Members]

| Member Name | Description |
| --- | --- |
| dev_max_num | The number of devices matched by the system. |
| pci_bus_address | The bus address of the PCI devices. |

#### 1.1.37.7 PCIe EP Device Mode

[Description]

Defines the PCIe EP device mode.

[Definition]

```
struct pcie_dev_mode {
    unsigned short mode;
    unsigned short submode;
};
```

[Members]

| Member Name | Description |
| --- | --- |
| mode | Current mode of the PCIe EP device, with the following value ranges:<br>#define RKEP_MODE_BOOTROM 1<br>#define RKEP_MODE_LOADER 2<br>#define RKEP_MODE_KERNEL 3<br>#define RKEP_MODE_FUN0 4<br>#define RKEP_MODE_ERR 0xffff //Device exception requires reload |
| submode | Status corresponding to the mode of the PCIe EP device, with the following value ranges:<br>#define RKEP_SMODE_INIT 0<br>#define RKEP_SMODE_LNKRDY 1<br>#define RKEP_SMODE_LNKUP 2<br>#define RKEP_SMODE_FWDLRDY 0x10<br>#define RKEP_SMODE_FWDLDONE 0x11<br>#define RKEP_SMODE_APPRDY 0x20<br>#define RKEP_SMODE_APPDEINIT 0x21<br>#define RKEP_SMODE_ERR 0xffff |

#### 1.1.37.8 pcie_dev_attr_st

[Description]

Defines the structure of PCIE EP device attributes.

[Definition]

```
struct pcie_dev_attr_st {
    int vendor_id;
    int device_id;
    char pci_bus_address[PCI_BUS_MAX_NAME];
    unsigned char using_rc_dma: 1;
    unsigned char rc_dma_chn: 1;
    unsigned char enable_speed: 1;
    unsigned char use_dma_only_in_rc: 1; // Only call DMA to transmit and receive data on the
RC side
};
```

[Members]

| Member Name | Description |
| --- | --- |
| vendor_id | Vendor ID of the EP device. |
| device_id | Device ID of the EP device. |
| pci_bus_address | Bus address of the EP device. |
| using_rc_dma | In the RK interconnection scenario, this mode can be enabled to use RC-side DMA to increase transmission rate. It is recommended to disable this configuration in multi-EP scenarios and use EP DMA. |
| rc_dma_chn | RC DMA supports 2 channels, with values 0/1. |
| enable_speed | Enable speed statistics function. The total transmission rate of the current EP device can be viewed through cat /tmp/pcie_debug*. |
| use_dma_only_in_rc | Only call DMA to transmit and receive data on the RC side. |

#### 1.1.37.9 PCIe Task Attribute Structure

[Description]

Defines the structure for PCIe Task attributes.

[Definition]

```
struct pcie_task_attr_st {
    int wait_timeout_ms;
    unsigned int send_buff_num;
    size_t send_buff_size;
    unsigned int recv_buff_num;
    size_t recv_buff_size;
};
```

[Members]

| Member Name | Description |
| --- | --- |
| wait_timeout_ms | Sets the interface timeout, -1 for blocking mode. |
| send_buff_num | Number of PCIE send buffers for buffer polling. |
| send_buff_size | Memory size of the send buffer. |
| recv_buff_num | Number of PCIE receive buffers for buffer polling. |
| recv_buff_size | Memory size of the receive buffer. |

#### 1.1.37.10 pcie_task_msg_st

[Description]

Defines the PCIE Task message structure.

[Definition]

```
struct pcie_task_msg_st {
    enum ETask_Msg_Type type;
    int task_id;
    unsigned int size;
    char lock;
    unsigned char data[TASK_MSG_DATA_SIZE];
};
```

[Members]

| Member Name | Description |
| --- | --- |
| type | Type of message. |
| task_id | Task ID. |
| size | Length of message. |
| lock | Reserved. |
| data | Content of message. |

#### 1.1.37.11 pcie_ep_obj_info_for_user

[Description]

Defines the PCIE basic information structure.

[Definition]

```
struct pcie_ep_obj_info_for_user {
    unsigned int magic;
    unsigned int rc_cc_version;
    unsigned int ep_cc_version;
    unsigned int func_drv_version;
    unsigned short vid;
    unsigned short did;
    unsigned short sub_vid;
    unsigned short sub_did;
};
```

[Members]

| Member Name | Description |
| --- | --- |
| magic | Chips Connect development package magic, 32-bit width, default value is 0x524B4550, assigned by EP kernel driver |
| rc_cc_version | RC Chips Connect development package version information, 32-bit width, incrementing count, for example 0x1 corresponds to V1 |
| ep_cc_version | EP Chips Connect development package version information, 32-bit width, incrementing count, for example 0x1 corresponds to V1 |
| func_drv_version | PCIe driver version information, 32-bit width, high eight bits redundant, the rest each 8 bits represent a version information, for example 0x00030100 corresponds to V3.1.0 |
| vid | PCI manufacturer ID, 16-bit width |
| did | PCI device ID, 16-bit width |
| sub_vid | Subsystem manufacturer ID, 16-bit width |
| sub_did | Subsystem device ID, 16-bit width |

#### 1.1.37.12 pcie_err_event_count

[Description]

Defines the PCIe error event count information structure.

[Definition]

```
struct pcie_err_event_count {
    unsigned char ebuf_overflow;
    unsigned char ebuf_under_run;
    unsigned char decode_error;
    unsigned char running_disparity_error;
    unsigned char skp_os_parity_error;
    unsigned char sync_header_error;
    unsigned char ctl_skp_os_parity_error;
    unsigned char detect_ei_infer;
    unsigned char receiver_error;
    unsigned char rx_recovery_request;
    unsigned char n_fts_timeout;
    unsigned char framing_error;
    unsigned char deskew_error;
    unsigned char bad_tlp;
    unsigned char lrc_error;
    unsigned char bad_dllp;
    unsigned char replay_number_rollover;
    unsigned char replay_timeout;
    unsigned char rx_nak_dllp;
    unsigned char tx_nak_dllp;
```

```
    unsigned char retry_tlp;
    unsigned char fc_timeout;
    unsigned char poisoned_tlp;
    unsigned char ecr_error;
    unsigned char unsupported_request;
    unsigned char completer_abort;
    unsigned char completion_timeout;
    /* The location above is hardware information and cannot be changed */
    unsigned char redundant_phase;
};
```

[Members]

| Member Name | Abbreviation | Description |
| --- | --- | --- |
| ebuf_overflow | EBUF Overflow | - |
| ebuf_under_run | EBUF Under-run | - |
| decode_error | Decode Error | Packet decoding error, signal anomaly |
| running_disparity_error | Running Disparity Error | Disparity error in 8b/10b encoding, Gen2 signal anomaly |
| skp_os_parity_error | SKP OS Parity Error | SKP sequence parity error, Gen3 signal anomaly |
| sync_header_error | SYNC Header Error | Synchronization header error, signal issue |
| ctl_skp_os_parity_error | CTL SKP OS Parity Error | Controlled SKP sequence parity error, Gen3 signal anomaly |
| detect_ei_infer | Detect EI Infer | Detected signal crosstalk |
| receiver_error | Receiver Error | Receiver error |
| rx_recovery_request | Rx Recovery Request | RX signal receives a peripheral request to enter recovery state for signal correction |
| n_fts_timeout | N_FTS Timeout | Peripheral's n_FTS does not meet the standard, L0s returns to L0 abnormally and enters recovery state |
| framing_error | Framing Error | Frame decoding error, signal anomaly |
| deskew_error | Deskew Error | Deskew error, signal anomaly |
| bad_tlp | BAD TLP | Received erroneous TLP packet |
| lrc_error | LCRC Error | Link CRC error, signal anomaly |
| bad_dllp | BAD DLLP | Erroneous DLLP packet, signal anomaly |
| replay_number_rollover | Replay Number Rollover | Accumulated retransmission packets overflow replay buffer |
| replay_timeout | Replay Timeout | Retransmission timeout for erroneous packets |

| Member Name | Abbreviation | Description |
| --- | --- | --- |
| rx_nak_dllp | Rx Nak DLLP | Received a DLLP from downstream and rejected its access |
| tx_nak_dllp | Tx Nak DLLP | DLLP sent to downstream was rejected |
| retry_tlp | Retry TLP | Number of retransmitted TLP packets, can indicate error rate |
| fc_timeout | FC Timeout | Flow control update timeout |
| poisoned_tlp | Poisoned TLP | Reported erroneous TLP type, possibly due to link issues or RAM bit flips |
| ecr_error | ECRC Error | End CRC error, signal anomaly |
| unsupported_request | Unsupported Request | Received an unsupported request type TLP packet, possibly due to denied peripheral access |
| completer_abort | Completer Abort | Device received a request from RC but encountered an internal error, requiring termination of the access request |
| completion_timeout | Completion Timeout | Timeout for reading peripheral-related resources, TLP return timeout |
| redundant_phase | Redundant Phase | Counting variable to determine if status information is continuously updating |

#### 1.1.37.13 pcie_replace_buff_info_st

[Description]

Defines the replace buff info structure, same as pcie_buff_node structure.

[Definition]

```
typedef struct pcie_buff_node pcie_replace_buff_info_st;
```

[Members]

Please refer to the pcie_buff_node member description.

#### 1.1.37.14 pcie_replace_buff_info_list_st

[Description]

Defines the replace buff info list structure for batch replacing buff addresses.

[Definition]

```
#define PCIE_REPLACE_BUFF_MAX_NUM       (7)

struct pcie_replace_buff_info_list_st {
    pcie_replace_buff_info_st info[PCIE_REPLACE_BUFF_MAX_NUM];
    unsigned int buff_num;
};
```

[Members]

| Member Name | Description |
| --- | --- |
| info | Replace buff info array, supports up to PCIE_REPLACE_BUFF_MAX_NUM |
| buff_num | Actual number of buffs to replace |

## 1.2 PCIE Windows EP

Windows drivers provide various ioctl functions for applications:

| ioctl | Function Description |
| --- | --- |
| IOCTL_RKEP_DOWNLOAD_FIRMWARE | Transfer firmware to EP |
| IOCTL_RKEP_BOOT_RUN | Initiate EP boot after firmware transfer |
| IOCTL_RKEP_CREATE_TASK | Create Task |
| IOCTL_RKEP_DESTROY_TASK | Release Task |
| IOCTL_RKEP_SEND_BUF | Send buffer to EP |
| IOCTL_RKEP_GET_BUF | Retrieve buffer data from EP |
| IOCTL_RKEP_GET_EP_MODE | Get the current mode of EP |
| IOCTL_RKEP_SEND_MSG_TO_EP_BUS | Send message event to EP |
| IOCTL_RKEP_GET_MSG_FOR_TASK_ID | Receive message event from EP |

### 1.2.1 Data Types

PCIE RC related data types and data structure definitions are as follows:

- RK_PCIE_TASK_ATTR: Defines the Task attribute structure.
- RK_PCIE_DESTROY_TASK: Releases the Task structure.
- RK_PCIE_IOCTL_STATUS: Defines the PCIE ioctl call status structure.
- RK_PCIE_TASK_MSG: Defines the Task message structure.
- RK_PCIE_BUFF_NODE: Defines the PCIE EP BUF structure.
- RK_PCIE_DEV_MODE: Defines the PCIE EP device mode structure.

- RK_PCIE_TASK_INFO: Defines the Task information structure.
- RK_PCIE_DOWNLOAD_FIRMWARE: Defines the firmware download structure.

#### 1.2.1.1 RK_PCIE_TASK_ATTR

[Description]

Defines the structure of PCIE Task attributes.

[Definition]

```
typedef struct _RK_PCIE_TASK_ATTR {
    int WaitTimeoutMs;
    int SendBuffNum;
    int SendBuffSize;
    int RecvBuffNum;
    int RecvBuffSize;
}  RK_PCIE_TASK_ATTR, *PRK_PCIE_TASK_ATTR;
```

[Members]

| Member Name | Description |
| --- | --- |
| WaitTimeoutMs | Sets the interface timeout, does not support blocking mode. |
| SendBuffNum | The number of PCIE send buffers, used for buffer polling. |
| SendBuffSize | The memory size of the send buffer. |
| RecvBuffNum | The number of PCIE receive buffers, used for buffer polling. |
| RecvBuffSize | The memory size of the receive buffer. |

#### 1.2.1.2 RK_PCIE_DESTROY_TASK

[Description]

Define the structure for releasing PCIE Task.

[Definition]

```
typedef struct _RK_PCIE_DESTROY_TASK {
    int TaskID;
}  RK_PCIE_DESTROY_TASK, *PRK_PCIE_DESTROY_TASK;
```

[Members]

| Member Name | Description |
| --- | --- |
| TaskID | The Task ID to be released |

#### 1.2.1.3 RK_PCIE_IOCTL_STATUS

[Description]

Defines the PCIE ioctl call status structure.

[Definition]

```
typedef struct _RK_PCIE_IOCTL_STATUS {
    int Status;
    char Data[0];
} RK_PCIE_IOCTL_STATUS, *PRK_PCIE_IOCTL_STATUS;
```

[Members]

| Member Name | Description |
| --- | --- |
| Status | The return value of the ioctl call, for details refer to the enum _RK_PCIE_RET list |
| Data | Data returned by ioctl |

#### 1.2.1.4 RK_PCIE_TASK_MSG

[Description]

Defines the Task message structure.

[Definition]

```
typedef struct _RK_PCIE_TASK_MSG {
    int TaskID;
    int Size;
    char Data[TASK_MSG_DATA_SIZE];
} RK_PCIE_TASK_MSG, *PRK_PCIE_TASK_MSG;
```

[Members]

| Member Name | Description |
| --- | --- |
| TaskID | Task ID |
| Size | Message length, maximum length is 1504 Bytes |
| Data | Message content |

#### 1.2.1.5 RK_PCIE_BUFF_NODE

[Description]

Defines the PCIE EP BUF structure.

[Definition]

```
typedef struct _RK_PCIE_BUFF_NODE {
    int Size;
    int TaskID;
    int TimeOut10Ms;
    char Reserved[52];  // 64 = 52 + 4 + 4 + 4
    char Data[0];
} RK_PCIE_BUFF_NODE, *PRK_PCIE_BUFF_NODE;
```

[Members]

| Member Name | Description |
| --- | --- |
| Size | Data length |
| TaskID | Task ID |
| TimeOut10Ms | Sets the interface timeout, does not support blocking mode. Minimum unit is 10ms |
| Reserved | Reserved size |
| Data | Data |

#### 1.2.1.6 RK_PCIE_DEV_MODE

[Description]

Defines the structure of the PCIE EP device mode.

[Definition]

```
typedef struct _RK_PCIE_DEV_MODE {
    unsigned short Mode;
    unsigned short SubMode;
} RK_PCIE_DEV_MODE, *PRK_PCIE_DEV_MODE;
```

[Members]

| Member Name | Description |
| --- | --- |
| Mode | Current mode of the PCIE EP device, with the following possible values:<br>#define RKEP_MODE_BOOTROM 1<br>#define RKEP_MODE_LOADER 2<br>#define RKEP_MODE_KERNEL 3<br>#define RKEP_MODE_FUN0 4<br>#define RKEP_MODE_ERR 0xffff //Device exception, reload required |
| SubMode | Status corresponding to the mode of the PCIE EP device, with the following possible values:<br>#define RKEP_SMODE_INIT 0<br>#define RKEP_SMODE_LNKRDY 1<br>#define RKEP_SMODE_LNKUP 2<br>#define RKEP_SMODE_FWDLRDY 0x10<br>#define RKEP_SMODE_FWDLDONE 0x11<br>#define RKEP_SMODE_APPRDY 0x20<br>#define RKEP_SMODE_APPDEINIT 0x21<br>#define RKEP_SMODE_ERR 0xffff |

#### 1.2.1.7 RK_PCIE_TASK_INFO

[Description]

Defines the Task information structure.

[Definition]

```
typedef struct _RK_PCIE_TASK_INFO {
    int TaskID;
} RK_PCIE_TASK_INFO, *PRK_PCIE_TASK_INFO;
```

[Members]

| Member Name | Description |
| --- | --- |
| TaskID | Task ID |

#### 1.2.1.8 RK_PCIE_DOWNLOAD_FIRMWARE

[Description]

Defines the firmware download structure.

[Definition]

```
typedef struct _RK_PCIE_DOWNLOAD_FIRMWARE {
    size_t RemoteLoadAddr;
    size_t Size;
    char Data[0];
} RK_PCIE_DOWNLOAD_FIRMWARE, *PRK_PCIE_DOWNLOAD_FIRMWARE;
```

[Members]

| Member Name | Description |
| --- | --- |
| RemoteLoadAddr | EP device memory address |
| Size | Firmware length |
| Data | Firmware content |

# 1.3 PCIE Endpoint

## 1.3.1 rk_pcie_set_log_level

[Description]

Set the log print level, default is debug.

[Syntax]

```
int rk_pcie_set_log_level(int level)
```

[Parameters]

| Parameter Name | Description | Input/Output |
| --- | --- | --- |
| level | Log level, options: 6(verbose), 5(debug), 4(info), 3(warn), 2(error), 1(fatal) | Input |

[Return Values]

| Return Value | Description |
| --- | --- |
| 0 | Success |
| Non-zero | Failure, please refer to Error Codes |

## 1.3.2 rk_pcie_set_log_cb

[Description]

Set log callback function to redirect logs from default console output to caller.

[Syntax]

```
int rk_pcie_set_log_cb(log_callback cb)
```

[Parameters]

| Parameter Name | Description | Input/Output |
| --- | --- | --- |
| cb | Log callback function pointer, type is `int (*log_callback)(char *log)` | Input |

[Return Values]

| Return Value | Description |
| --- | --- |
| 0 | Success |
| Non-zero | Failure, please refer to Error Codes |

### 1.3.3 rk_pcie_device_init

[Description]

Initializes the PCIe device

[Syntax]

```
int rk_pcie_device_init(void)
```

[Parameters]

None

[Return Values]

| Return Value | Description |
| --- | --- |
| 0 | Success |
| Non-zero | Failure, see Error Codes |

[Example]

```
typedef struct
{
    pthread_t thr_recv_rc;
    pthread_t thr_send_rc;
```

```
    char recv_rc_exit;
    char send_rc_exit;
} ep_context_st;

int main(void)
{
    int ret = 0;
    struct task_context_st task_ctx[PCIE_DMA_TASK_MAX] = {0};
    struct pcie_task_msg_st msg;
    struct task_user_cmd_st *user_cmd;

    ret = rk_pcie_device_init();
    if (ret != 0) {
        printf("pcie init device error....\n");
        return -1;
    }

    while (1) {
        ret = rk_pcie_get_msg_for_bus(&msg, 100);
        if (ret) {
            printf("task_id:%d type:%d size:%ld \n", msg.task_id, msg.type, msg.size);
            switch (msg.type) {
            case E_TASK_MSG_CREATE:
                task_create(&task_ctx[msg.task_id], msg.task_id);
                break;
            case E_TASK_MSG_DESTROY:
                task_destroy(&task_ctx[msg.task_id], msg.task_id);
                break;
            case E_TASK_MSG_USER: {
                user_cmd = (struct task_user_cmd_st *)msg.data;
                switch (user_cmd->cmd) {
                case TASK_USER_CMD_INIT_APP:
                    app_create(&task_ctx[msg.task_id], msg.task_id);
                    break;
                case TASK_USER_CMD_DEINIT_APP:
                    app_destroy(&task_ctx[msg.task_id], msg.task_id);
                    break;
                }
            }
            break;
            default:
                printf("error msg type!!! \n");
                break;
            }
        }
    }

    rk_pcie_device_deinit();

    return 0;
}
```

[Related Topics]

rk_pcie_device_deinit

### 1.3.4 rk_pcie_device_deinit

[Description]

Deinitialize PCIe, release resources

[Syntax]

```
int rk_pcie_device_deinit(void)
```

### 1.3.5 rk_pcie_task_create

[Description]

Create a Task and initialize memory and BUFF management.

[Syntax]

```
int rk_pcie_task_create(int task_id)
```

[Parameters]

| Parameter Name | Description | Input/Output |
| --- | --- | --- |
| task_id | Task ID, passed by message from RC end | Input |

[Return Values]

| Return Value | Description |
| --- | --- |
| 0 | Success |
| Non-zero | Failure, please refer to Error Codes |

[Example]

Please refer to the example in rk_pcie_device_init.

[Related Topics]

rk_pcie_device_deinit

### 1.3.6 rk_pcie_task_destroy

[Description]

Deinitialize the Task, releasing memory, BUFF management, and other resources.

[Syntax]

```
int rk_pcie_task_destroy(int task_id)
```

[Parameters]

| Parameter Name | Description | Input/Output |
| --- | --- | --- |
| task_id | Task ID | Input |

[Return Values]

| Return Value | Description |
| --- | --- |
| 0 | Success |
| Non-zero | Failure, see Error Codes |

[Example]

Please refer to the example in rk_pcie_device_init.

[Related Topics]

rk_pcie_device_init

### 1.3.7 rk_pcie_set_ep_info

[Description]

Set EP status information.

[Syntax]

```
int rk_pcie_set_ep_info(void *status, size_t status_size)
```

[Parameters]

| Parameter Name | Description | Input/Output |
| --- | --- | --- |
| status | void pointer, status structure defined by user | Input |
| status_size | size of the status structure | Input |

[Return Values]

| Return Value | Description |
| --- | --- |
| 0 | Success |
| Non-zero | Failure, see Error Codes |

[Example]

```
struct ep_info_st {
    unsigned int link_status;
    unsigned int temp;
    long long timestamp;
    unsigned char cpu_load;
    unsigned char npu_load;
    unsigned char gpu_load;
};

ep_info_st ep_info;
while(1) {
    ep_info.timestamp = time(NULL);
    rk_pcie_set_ep_info((void *)&ep_info, sizeof(struct ep_info_st));
}
```

[Related Topics]

rk_pcie_get_ep_info

### 1.3.8 rk_pcie_get_user_cmd

[Description]

Retrieve a memory pointer at a specific location in BAR0 for user-defined command areas.

[Syntax]

```
void *rk_pcie_get_user_cmd(void)
```

[Parameters]

None

[Return Value]

| Return Value | Description |
| --- | --- |
| Non-NULL | Normal, pointer to custom command area memory |
| NULL | Failure |

[Example]

```
struct user_cmd_st {
    unsigned int max_chn;
    unsigned int width;
    unsigned int height;
    unsigned int codecid;
    unsigned int output_id;
    unsigned int init;
};

ep_ctx.user_cmd = rk_pcie_get_user_cmd();

while(ep_ctx.user_cmd->init != 1) {
    usleep(1000);
}

ep_ctx.video_obj.width = ep_ctx.user_cmd->width;
ep_ctx.video_obj.height = ep_ctx.user_cmd->height;
ep_ctx.video_obj.input_mode = ep_ctx.user_cmd->codecid;
ep_ctx.video_obj.output_mode = ep_ctx.user_cmd->output_id;
ep_ctx.video_obj.max_chn = ep_ctx.user_cmd->max_chn;
ep_ctx.max_chn = ep_ctx.video_obj.max_chn;
```

[Related Topics]

rk_pcie_get_user_cmd

### 1.3.9 rk_pcie_get_bar1_user_buffer

[Description]

Obtain the virtual mapping address of bar1.

[Syntax]

```
void *rk_pcie_get_bar1_user_buffer(void)
```

[Parameters]

None.

[Return Value]

| Return Value | Description |
| --- | --- |
| void * | Virtual address |
| NULL | Failure, please refer to Error Codes. |

### 1.3.10 rk_pcie_get_bar5_user_buffer

[Description]

Obtains the virtual mapping address of bar5.

[Syntax]

```
void *rk_pcie_get_bar5_user_buffer(void)
```

[Parameters]

None.

[Return Value]

| Return Value | Description |
| --- | --- |
| void * | Virtual address |
| NULL | Failure, see Error Codes. |

### 1.3.11 rk_pcie_set_bar_inbound_cpu_addr

[Description]

Set BAR inbound mapping.

[Syntax]

```
u_int64_t rk_pcie_set_bar_inbound_cpu_addr(u_int32_t atu_index, u_int64_t cpu_address,
u_int32_t size)
```

[Parameters]

| Parameter Name | Description | Input/Output |
| --- | --- | --- |
| atu_index | ATU index | Input |
| cpu_address | Mapped CPU address | Input |
| size | Mapping size limit | Input |

[Return Value]

| Return Value | Description |
| --- | --- |
| cpu_address | Returns CPU address |

### 1.3.12 rk_pcie_get_buff

[Description]

Obtain an available PCIE buffer.

[Syntax]

```
struct pcie_buff_node *rk_pcie_get_buff(int task_id, enum EBuff_Type type)
```

[Parameters]

| Parameter Name | Description | Input/Output |
| --- | --- | --- |
| task_id | Task ID | Input |
| type | PCIE Buff Type | Input |

[Return Values]

| Return Value | Description |
| --- | --- |
| Non-NULL | Pointer to available Buff |
| NULL | No available Buff |

[Example]

```
node = rk_pcie_get_buff(ctx->task_id, E_SEND);
if (node) {
    node->size = buffer_size;
    memset(node->vir_addr, index, node->size);
    add_check_info(node->vir_addr + node->size - CHECK_SIZE);
    rk_pcie_send_buff(ctx->task_id, node);
    index++;
} else {
    usleep(1000);
}
```

[Related Topics]

rk_pcie_send_buff

### 1.3.13 rk_pcie_send_buff

[Description]

Sending PCIE buffer.

[Syntax]

```
int rk_pcie_send_buff(int task_id, struct pcie_buff_node *pbuff)
```

[Parameters]

| Parameter Name | Description | Input/Output |
| --- | --- | --- |
| task_id | Task ID | Input |
| pbuff | PCIE Buffer Pointer | Input |

[Return Values]

| Return Value | Description |
| --- | --- |
| 0 | Successfully sent |
| Non-zero | Failure, see Error Codes |

[Example]

rk_pcie_get_buff

[Related Topics]

rk_pcie_get_buff

### 1.3.14 rk_pcie_release_buff

[Description]

Releases the PCIE buffer.

[Syntax]

```
int rk_pcie_release_buff(int task_id, struct pcie_buff_node *pbuff)
```

[Parameters]

| Parameter Name | Description | Input/Output |
| --- | --- | --- |
| task_id | Task ID | Input |
| pbuff | PCIE Buff Pointer | Input |

[Return Values]

| Return Value | Description |
| --- | --- |
| 0 | Successfully released |
| Non-zero | Failure, see Error Codes |

[Example]

```
node = rk_pcie_get_buff(ctx->task_id, E_RECV);
if (node) {
    if (check_info(node->vir_addr + node->size - CHECK_SIZE) == 0) {
        pthread_mutex_lock(&ctx->mutex_stream_speed);
        ctx->stream_speed_index++;
        pthread_mutex_unlock(&ctx->mutex_stream_speed);
        // if (mptr != NULL) {
        //     memcpy(mptr, node->vir_addr, node->size);
        // }
        memset(node->vir_addr + node->size - CHECK_SIZE, 0, CHECK_SIZE);
    } else {
        printf("check data ==================== error \n");
        for (int i = 0; i < CHECK_SIZE; i++) {
            unsigned char *p = node->vir_addr + node->size - CHECK_SIZE;
            printf("%d ", p[i]);
        }
        printf("\ncheck data ===ssss========node->size = %d======== error \n", (int)node-
>size);
    }
    rk_pcie_release_buff(ctx->task_id, node);
} else {
    usleep(1000);
}
```

### 1.3.15 rk_pcie_get_buff_max_size

[Description]

Obtain the maximum supported size of the buffer.

[Syntax]

```
long rk_pcie_get_buff_max_size(int task_id, enum EBuff_Type type)
```

[Parameters]

| Parameter Name | Description | Input/Output |
| --- | --- | --- |
| task_id | Task ID | Input |
| type | PCIE Buffer Type | Input |

[Return Values]

| Return Value | Description |
| --- | --- |
| Greater than or equal to 0 | buffer size |
| Less than 0 | Failure, please refer to Error Codes |

[Example]

```
long buffer_size = rk_pcie_get_buff_max_size(ctx->task_id, E_SEND);
```

### 1.3.16 rk_pcie_get_msg_for_bus

[Description]

Wait for the RC-side message.

[Syntax]

```
int rk_pcie_get_msg_for_bus(struct pcie_task_msg_st *msg, int timeout_ms)
```

[Parameters]

| Parameter Name | Description | Input/Output |
| --- | --- | --- |
| msg | Pointer to message structure | Output |
| timeout_ms | Timeout period, -1 for blocking mode (not recommended, will wait indefinitely in the kernel), it is recommended to use 100ms, unit ms | Input |

[Return Values]

| Return Value | Description |
| --- | --- |
| 0 | Success |
| Non-zero | Failure, please refer to Error Codes |

[Example]

Please refer to the example in rk_pcie_device_init.

[Related Topics]

rk_pcie_device_deinit

### 1.3.17 rk_pcie_send_msg_to_rc_task

[Description]

Send a message to the RC side

[Syntax]

```
int rk_pcie_send_msg_to_rc_task(int task_id, void *data, size_t size)
```

[Parameters]

| Parameter Name | Description | Input/Output |
| --- | --- | --- |
| task_id | Task ID | Input |
| data | Message Pointer | Input |
| size | Message Length | Input |

[Return Value]

| Return Value | Description |
| --- | --- |
| 0 | Success |
| Non-zero | Failure, please refer to Error Codes |

[Example]

Please refer to the example in rk_pcie_device_init.

[Related Topics]

rk_pcie_device_deinit

### 1.3.18 rk_pcie_check_buff_avail

[Description]

Check if there is a valid recv buff available.

[Syntax]

```
int rk_pcie_check_buff_avail(int task_id)
```

[Parameters]

| Parameter Name | Description | Input/Output |
| --- | --- | --- |
| task_id | Task ID | Input |

[Return Values]

| Return Value | Description |
| --- | --- |
| cc_true | Valid buff exists |
| cc_false | No valid buff |

### 1.3.19 rk_pcie_get_err_event_count

[Description]

Get PCIe error event count.

[Syntax]

```
int rk_pcie_get_err_event_count(int task_id, struct pcie_err_event_count *info)
```

[Parameters]

| Parameter Name | Description | Input/Output |
| --- | --- | --- |
| task_id | Task ID | Input |
| info | PCIe error event count | Output |

[Return Values]

| Return Value | Description |
| --- | --- |
| 0 | Success |
| Non-zero | Failure, please refer to Error Codes |

### 1.3.20 rk_pcie_is_alive

[Description]

Check if PCIe link is alive.

[Syntax]

```
int rk_pcie_is_alive(int task_id)
```

[Parameters]

| Parameter Name | Description | Input/Output |
| --- | --- | --- |
| task_id | Task ID | Input |

[Return Values]

| Return Value | Description |
| --- | --- |
| cc_true | Link is alive |
| cc_false | Link is down |

### 1.3.21 rk_pcie_replace_buff_all

[Description]

Replace all buff addresses for configuring user-defined buffs.

[Syntax]

```
int rk_pcie_replace_buff_all(int task_id, enum EBuff_Type type, struct
pcie_replace_buff_info_list_st *infos)
```

[Parameters]

| Parameter Name | Description | Input/Output |
| --- | --- | --- |
| task_id | Task ID | Input |
| type | Buff type (E_SEND or E_RECV) | Input |
| infos | Replace buff info list | Input |

[Return Values]

| Return Value | Description |
| --- | --- |
| 0 | Success |
| Non-zero | Failure, please refer to Error Codes |

[Related Topics]

rk_pcie_replace_buff pcie_replace_buff_info_list_st

### 1.3.22 rk_pcie_replace_buff

[Description]

Replace single buff address for configuring user-defined buffs.

[Syntax]

```
int rk_pcie_replace_buff(int task_id, enum EBuff_Type type, struct pcie_buff_node *pbuff,
pcie_replace_buff_info_st *info)
```

[Parameters]

| Parameter Name | Description | Input/Output |
| --- | --- | --- |
| task_id | Task ID | Input |
| type | Buff type (E_SEND or E_RECV) | Input |
| pbuff | Original buff node pointer | Input |
| info | New buff info | Input |

[Return Values]

| Return Value | Description |
| --- | --- |
| 0 | Success |
| Non-zero | Failure, please refer to Error Codes |

[Related Topics]

rk_pcie_replace_buff_all pcie_replace_buff_info_st

## 1.3.23 Data Types

PCIE RC-related data types and data structure definitions are as follows:

- EBuff_Type: Defines the PCIe Buffer type.
- ETask_Msg_Type: Defines the Task message type.
- struct pcie_buff_node: Defines the PCIe buffer attribute structure.
- struct pcie_task_msg_st: Defines the message attribute structure.

### 1.3.23.1 EBuff_Type

[Description]

Defines the types of PCIE Buff.

[Definition]

```
enum EBuff_Type {
    E_SEND = 0,
    E_RECV,
};
```

[Members]

| Member Name | Description |
| --- | --- |
| E_SEND | Send buffer. |
| E_RECV | Receive buffer. |

#### 1.3.23.2 ETask_Msg_Type

[Description]

Defines the types of task messages.

[Definition]

```
enum ETask_Msg_Type {
    E_TASK_MSG_NONE = 0,
    E_TASK_MSG_CREATE,
    E_TASK_MSG_DESTROY,
    E_TASK_MSG_USER,
    E_TASK_MSG_MAX,
};
```

[Members]

| Member Name | Description |
| --- | --- |
| E_TASK_MSG_NONE | Default value, no significance |
| E_TASK_MSG_CREATE | Create task |
| E_TASK_MSG_DESTROY | Destroy task |
| E_TASK_MSG_USER | User-defined message |
| E_TASK_MSG_MAX | Maximum message type |

#### 1.3.23.3 PCIe Buffer Node

[Description]

Defines the PCIe buffer attribute structure. Any modifications to this structure require changes to both RC and EP.

[Definition]

```
struct pcie_buff_node {
    void *vir_addr;
    size_t phy_addr;
    size_t size;
};
```

[Members]

| Member Name | Description |
| --- | --- |
| vir_addr | Virtual address. |
| phy_addr | Physical address. |
| size | Memory size.<br>Send buffer: Users need to configure the data size, which should not exceed the total buffer memory size.<br>Receive buffer: The size of the data sent by the counterparty. |

#### 1.3.23.4 pcie_task_msg_st

[Description]

Defines the message attribute structure. This structure needs to be modified simultaneously in both RC and EP.

[Definition]

```
struct pcie_task_msg_st {
    enum ETask_Msg_Type type;
    int task_id;
    unsigned int size;
    char lock;
    unsigned char data[TASK_MSG_DATA_SIZE];
};
```

[Members]

| Member Name | Description |
| --- | --- |
| type | Type of message. |
| task_id | Task ID. |
| size | Length of message. |
| lock | Reserved. |
| data | Content of message. |

## 1.4 Error Codes

| Error Code | Macro Definition | Description |
| --- | --- | --- |
| -1 | RK_PCIE_ERR_BAD | Abnormal PCIE Link |
| -2 | RK_PCIE_ERR_UNKNOWN | System Exception |
| -3 | RK_PCIE_ERR_NULL_PTR | Null Pointer Error |
| -4 | RK_PCIE_ERR_MALLOC | Memory Allocation Failure |
| -5 | RK_PCIE_ERR_OPEN_FILE | File Open Failure |
| -6 | RK_PCIE_ERR_VALUE | Error Value |
| -8 | RK_PCIE_ERR_TIMEOUT | Timeout Return |
| -10 | RK_PCIE_ERR_UNSUPPORT | Unsupported Feature |
| -13 | RK_PCIE_ERR_HW_UNSUPPORT | Hardware Unsupported |
| -15 | RK_PCIE_ERR_MMAP | Mmap Failure |
| -65 | RK_PCIE_ERR_INIT | Initialization Failure |
| -66 | RK_PCIE_ERR_NOINIT | System Not Initialized |
| -68 | RK_PCIE_ERR_NOMEM | Insufficient Memory |
| -69 | RK_PCIE_ERR_OUTOF_RANGE | Parameter Out of Range |